# JOB SHEET 2
# INTEROPERABILITAS
# OpenAPI, gRPC, dan WebSocket

Dosen Pengampu :

Wiktasari, S.T., M.Kom.

Disusun Oleh :

Muhammad Raafi Hariyadi

## I. Tujuan Instruksional Khusus

1. Memahami konsep interoperabilitas sistem menggunakan API dalam komunikasi antar aplikasi.
2. Menjelaskan konsep dan fungsi OpenAPI sebagai standar dokumentasi REST API.
3. Memahami struktur spesifikasi OpenAPI yang terdiri dari info, servers, paths, responses, dan components.
4. Menulis spesifikasi OpenAPI dalam format YAML atau JSON untuk mendokumentasikan layanan API.
5. Menggunakan Swagger UI untuk menampilkan dokumentasi API secara interaktif.
6. Mengimpor spesifikasi OpenAPI ke dalam Postman untuk melakukan pengujian API.
7. Memahami konsep gRPC (Remote Procedure Call) sebagai mekanisme komunikasi antar layanan.
8. Menjelaskan peran Protocol Buffers (Protobuf) dalam proses serialisasi data pada gRPC.
9. Menjelaskan jenis komunikasi pada gRPC seperti Unary RPC, Server Streaming, Client Streaming, dan Bidirectional Streaming.
10. Memahami konsep WebSocket sebagai protokol komunikasi dua arah (full-duplex) antara client dan server.
11. Menjelaskan perbedaan komunikasi HTTP request-response dengan komunikasi WebSocket real-time.
12. Membuat aplikasi sederhana menggunakan WebSocket untuk komunikasi data secara real-time.

## II. Landasan Teori

## III. Peralatan yang Digunakan

1. PC
2. Peramban web : Internet Explorer, Firefox, Google Chrome, Opera atau Safari
3. Aplikasi server local : XAMPP
4. Aplikasi text editor : Visual Studio Code

## IV. Langkah Kerja

4. 1. Install Node.js

1. Node.js digunakan untuk menjalankan server JavaScript dan membuat aplikasi backend seperti WebSocket dan gRPC.
2. Langkah Instalasi
   a. Buka website resmi
      https://nodejs.org
   b. Download **Node.js LTS (Long Term Support)**.
   c. Jalankan file installer yang telah diunduh.
   d. Ikuti langkah instalasi hingga selesai.
   e. Cek instalasi melalui terminal / command prompt:

```bash
node -v
npm -v
```

   Jika muncul nomor versi, maka instalasi berhasil.

4. 2. Install Postman

1. Postman digunakan untuk **menguji API** dan dapat mengimpor spesifikasi **OpenAPI** secara langsung.
2. Langkah Instalasi
   a. Buka websitehttps://www.postman.com/downloads/
   b. Download Postman sesuai sistem operasi.
   c. Install seperti aplikasi biasa.
   d. Jalankan Postman dan buat akun (opsional).

4. 3. Install Docker (Opsional untuk Swagger UI)

1. Install Docker (Opsional untuk Swagger UI)
2. Langkah Instalasi
   a. Buka website : https://www.docker.com/products/docker-desktop/
   b. Download Docker Desktop
   c. Install hingga selesai
   d. Cek instalasi dengan perintah:

```bash
docker --version
```

4. 4. Install Visual Studio Code

1. Visual Studio Code digunakan sebagai **code editor** untuk menulis kode OpenAPI, gRPC, dan WebSocket.

4. 5. Install OpenAPI Generator (Opsional)

1. Digunakan untuk menghasilkan **kode server atau client otomatis dari spesifikasi OpenAPI**.
2. Install menggunakan npm:

```bash
npm install @openapitools/openapi-generator-cli -g
```

3. Cek instalasi:

```bash
openapi-generator-cli version
```

4. 6. Install Library Praktikum

1. Mahasiswa perlu menginstall beberapa library Node.js.
2. WebSocket

```bash
npm install ws
```

3. gRPC

```bash
npm install @grpc/grpc-js
npm install @grpc/proto-loader
```

4. 7. PRAKTIKUM 1

Sebuah sistem **E-Learning** memiliki layanan API untuk mengelola data mahasiswa.

API memiliki endpoint berikut:

| Endpoint | Method | Fungsi |
|---|---|---|
| /students | GET | Mengambil daftar mahasiswa |
| /students | POST | Menambahkan mahasiswa |
| /students/{id} | GET | Melihat detail mahasiswa |

1. Membuat Folder Proyek
   a. Buat folder baru bernama:

```
openapi-praktikum
```

b. Masuk ke folder tersebut.

2. Membuat File Spesifikasi OpenAPI
   a. Buat file bernama:

```
openapi.yaml
```

   b. Isi dengan kode berikut:

```yaml
openapi: 3.0.0
info:
  title: Student API
  description: API untuk manajemen mahasiswa
  version: 1.0.0

servers:
  - url: http://localhost:3000

paths:
  /students:
    get:
      summary: Mengambil daftar mahasiswa
      responses:
        '200':
          description: Berhasil

    post:
      summary: Menambahkan mahasiswa
      responses:
        '201':
          description: Mahasiswa berhasil ditambahkan

  /students/{id}:
    get:
      summary: Mengambil detail mahasiswa
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Detail mahasiswa
```

3. Menjalankan Swagger UI menggunakan Docker

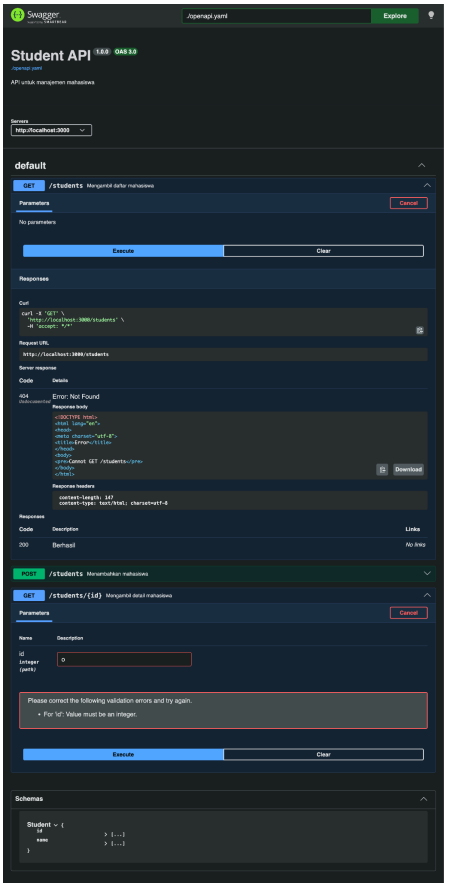
a. Jalankan perintah berikut:

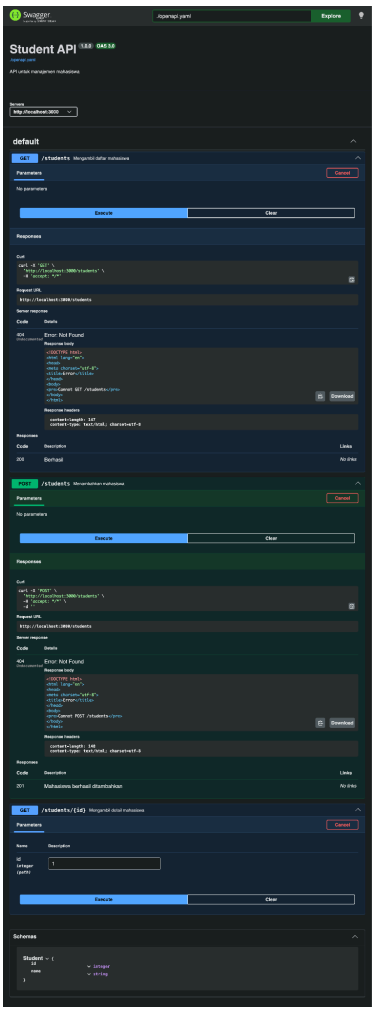
```bash
docker run -p 8080:8080 \
-e SWAGGER_JSON=/api/openapi.yaml \
-v $(pwd)/openapi.yaml:/api/openapi.yaml \
swaggerapi/swagger-ui
```

4. Membuka Dokumentasi API
   a. Buka browser :

```
http://localhost:8080
```

   b. Hasil yang akan muncul **(tampilkan masing-masing screenshot-nya)**
      - Dokumentasi API interaktif
      - Endpoint dapat diuji langsung
      - Struktur request dan response terlihat jelas

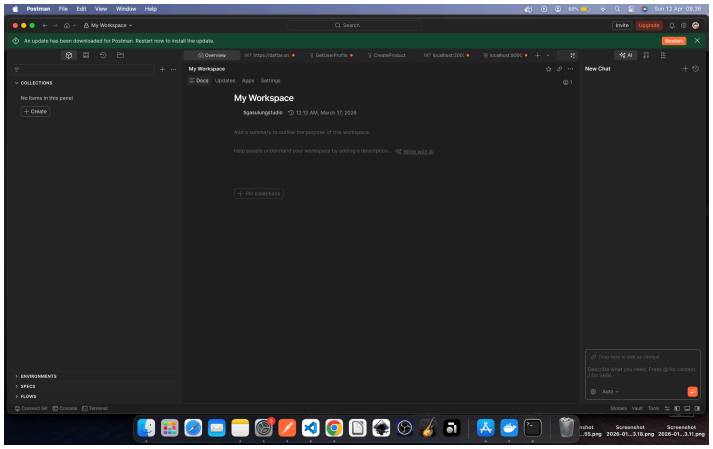




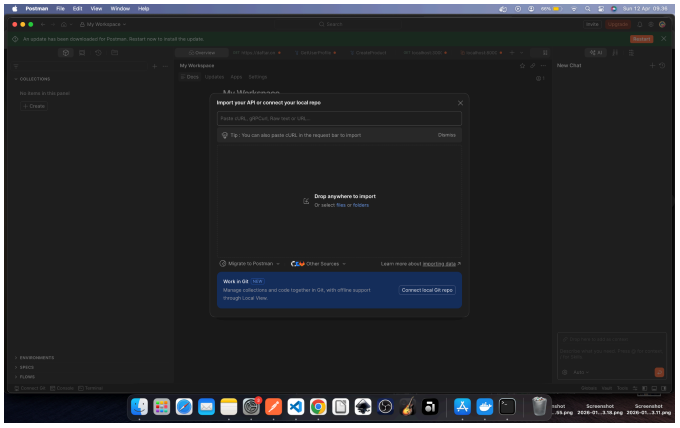

5. Menguji API menggunakan Postman

   Langkah **(tampilkan masing-masing screenshot-nya)**

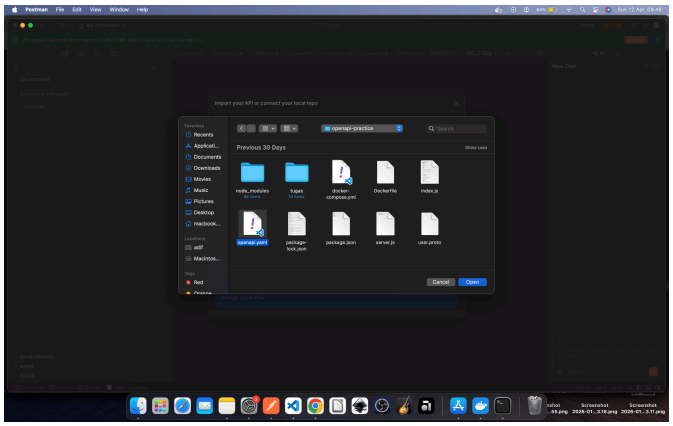
   a. Buka **Postman**



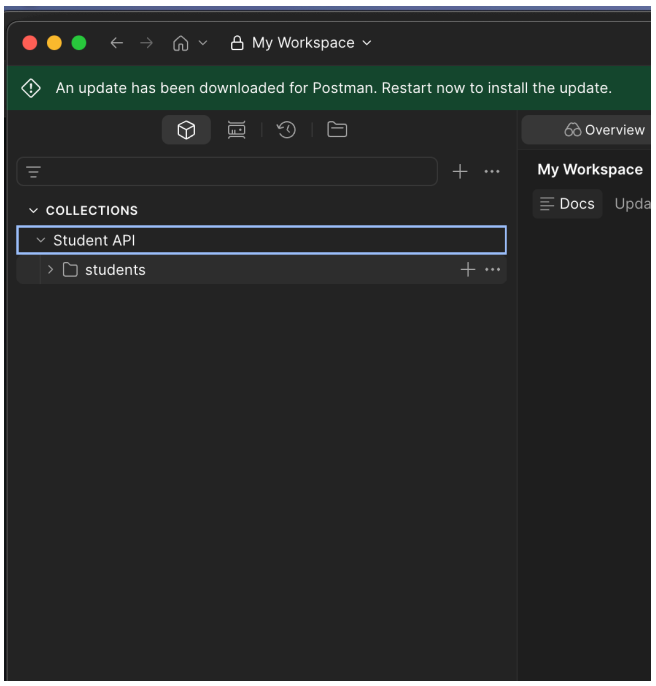

   b. Pilih **Import**



c. Upload file **openapi.yaml**



d. Postman akan otomatis membuat koleksi API




4. 8. PRAKTIKUM 2 (Konsep gRPC)

1. Studi Kasus : Sebuah sistem microservices memiliki layanan User Service yang menyediakan fungsi:

```
GetUserProfile
```

2. Membuat file Proto

Buat file:

```
user.proto
```

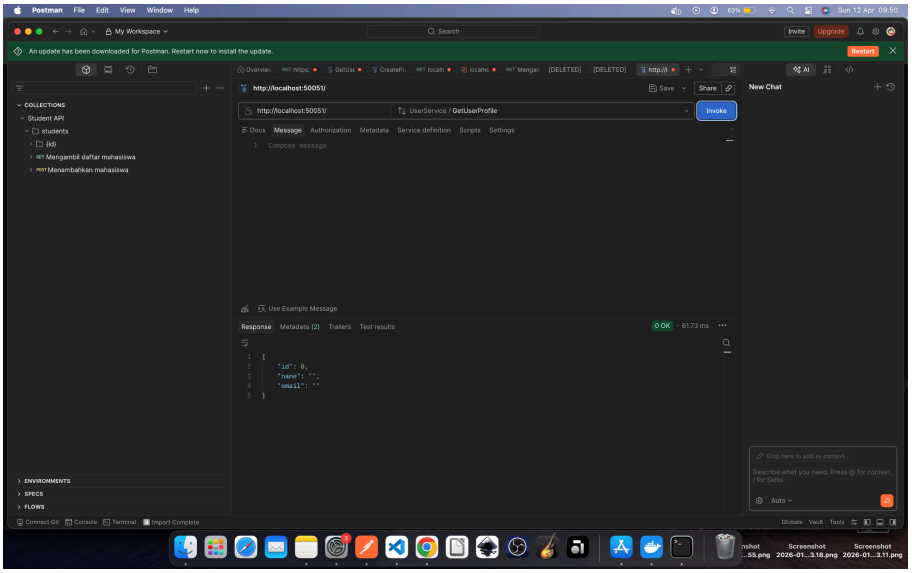
3. Isi dengan kode berikut:

```proto
syntax = "proto3";

service UserService {
  rpc GetUserProfile (UserRequest) returns (UserResponse);
}

message UserRequest {
  int32 user_id = 1;
}

message UserResponse {
  int32 id = 1;
  string name = 2;
  string email = 3;
}
```

4. Generate kode gRPC

```
npm install @grpc/grpc-js
npm install @grpc/proto-loader
```

5. Jalankan Server gRPC : Server akan menerima request dari client dan mengirim response **(berikan screenshot hasilnya)**




4. 9. PRAKTIKUM 1 (Implementasi WebSocket)

1. Studi Kasus : Buat aplikasi chat sederhana menggunakan WebSocket.
2. Install library :

```
npm install ws
```

3. Membuat Server WebSocket, buat file :

server.js

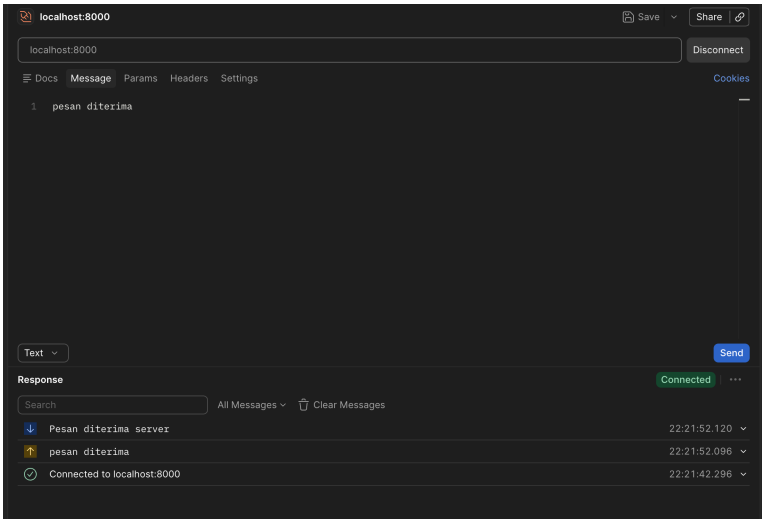
JavaScript

```javascript
const WebSocket = require('ws');

const server = new WebSocket.Server({ port: 3000 });

server.on('connection', socket => {

    socket.on('message', message => {
        console.log("Pesan diterima:", message);
        socket.send("Pesan diterima server");
    });

});
```

4. Jalankan Server :

```
node server.js
```

5. Uji menggunakan browser atau client WebSocket (**(berikan screenshot hasilnya)**


localhost:8000
Save
Share
localhost:8000
Disconnect
Docs
Message
Params
Headers
Settings
Cookies
1 pesan diterima
Text
Send
Response
Connected
Search
All Messages
Clear Messages
Pesan diterima server
22:21:52.120
pesan diterima
22:21:52.096
Connected to localhost:8000
22:21:42.296

## V. Lembar Kerja

1. Tugas 1

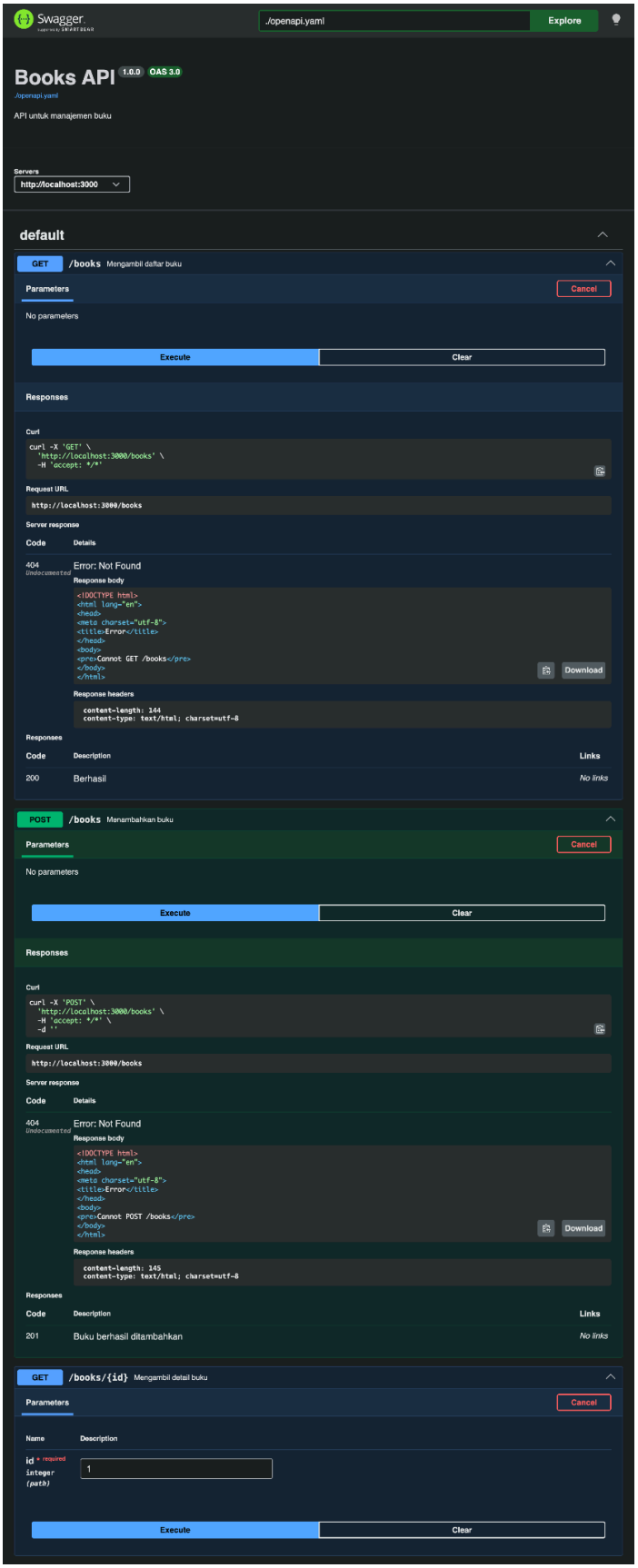
   Buat spesifikasi **OpenAPI untuk sistem perpustakaan** dengan endpoint berikut:

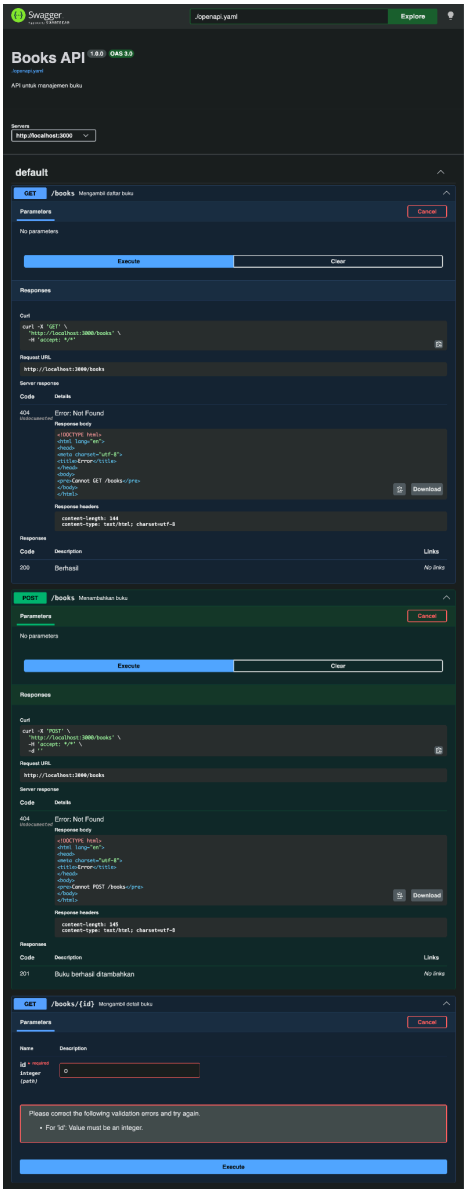
| **Endpoint** | **Method** |
|---|---|
| /books | GET |
| /books | POST |
| /books/{id} | GET |

Syntax:

```yaml
openapi: 3.0.0
info:
  title: Books API
  description: API untuk manajemen buku
  version: 1.0.0

servers:
  - url: http://localhost:3000

paths:
  /books:

    get:summary: Mengambil daftar buku
      responses:
        '200':
          description: Berhasil
    post:
      summary: Menambahkan buku
      responses:
        '201':
          description: Buku berhasil ditambahkan
  /books/{id}:
    get:
     summary: Mengambil detail buku
     parameters:
       - name: id
         in: path
         required: true
         schema:
           type: integer
    responses:
      '200':
        description: Detail buku
```

Hasil :


Swagger
./openapi.yaml
Explore
Books API 1.0.0 OAS 3.0
./openapi.yaml
API untuk manajemen buku
Servers
http://localhost:3000
default
GET /books Mengambil daftar buku
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:3000/books' \
-H 'accept: */*'
Request URL
http://localhost:3000/books
Server response
Code
Details
404
Undocumented
Error: Not Found
Response body
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /books</pre>
</body>
</html>
Download
Response headers
content-length: 144
content-type: text/html; charset=utf-8
Responses
Code
Description
Links
200
Berhasil
No links
POST /books Menambahkan buku
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://localhost:3000/books' \
-H 'accept: */*' \
-d ''
Request URL
http://localhost:3000/books
Server response
Code
Details
404
Undocumented
Error: Not Found
Response body
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot POST /books</pre>
</body>
</html>
Download
Response headers
content-length: 145
content-type: text/html; charset=utf-8
Responses
Code
Description
Links
201
Buku berhasil ditambahkan
No links
GET /books/{id} Mengambil detail buku
Parameters
Cancel
Name
Description
id * required
integer
(path)
1
Execute
Clear




2. Tugas 2

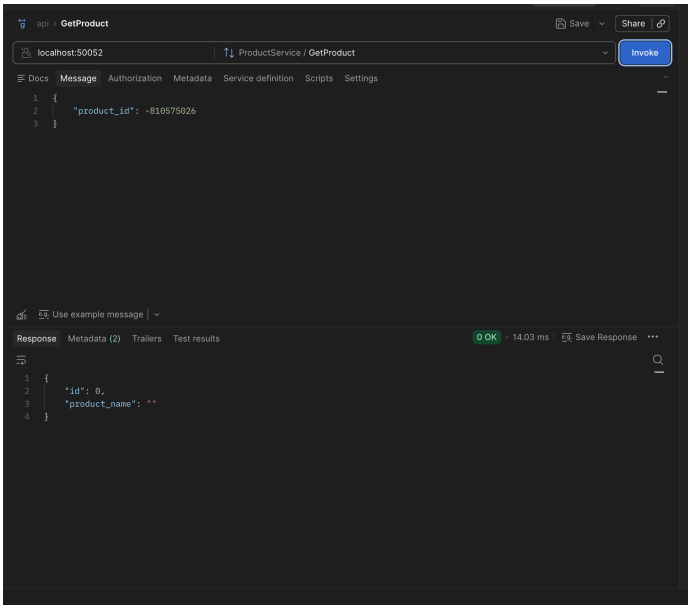
Buat skema **gRPC untuk layanan produk** dengan fungsi:

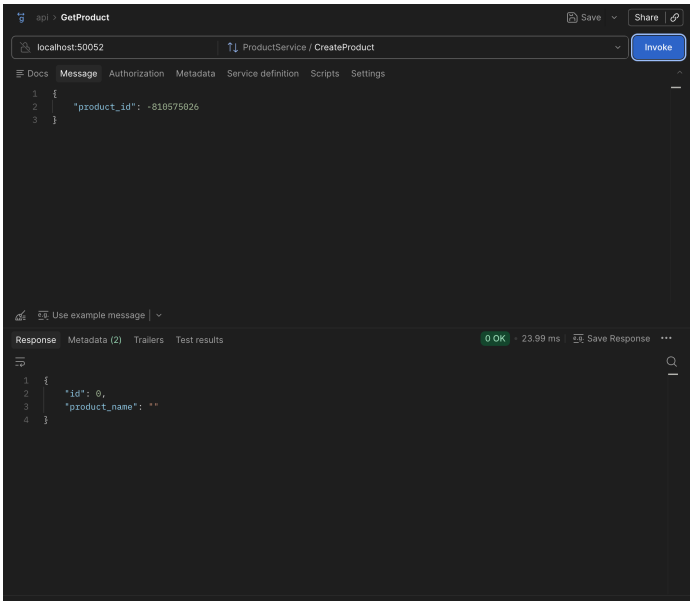
```
GetProduct
CreateProduct
```

Syntax :

```
syntax = "proto3";

package api;

service ProductService {
    rpc GetProduct(UserRequest) returns (UserResponse);
    rpc CreateProduct(CreateProductRequest) returns (UserResponse);
 }
message UserRequest {
    int32 product_id = 1;
 }

message CreateProductRequest {
    string product_name = 1;
 }

message UserResponse {
    int32 id = 1;
    string product_name = 2;
 }
```

Hasil :






3. Tugas 3

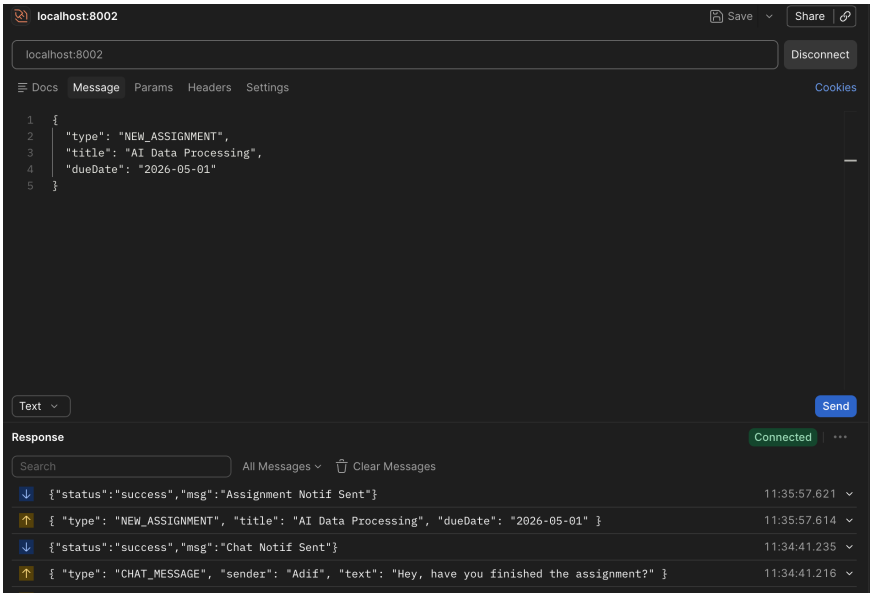
Buat aplikasi **WebSocket sederhana untuk notifikasi real-time** untuk

notifikasi tugas baru
notifikasi chat

Syntax :

```
const wss = new WebSocket.Server({ host: '0.0.0.0', port: 8002 });
wss.on('connection', socket => {
    console.log("Client linked for notifications");


    socket.on('message', data =>{
        try{const payload = JSON.parse(data);

            switch(payload.type) {
                case 'NEW_ASSIGNMENT':
                    console.log(`Notification: New Task - ${payload.title}`);
                    // Logic to save to DB or broadcast would go here
                    socket.send(JSON.stringify({ status: "success", msg: "Assignment Notif Sent" }));
                    break;

                case 'CHAT_MESSAGE':
                    console.log(`Notification: Chat from ${payload.sender}: ${payload.text}`);
                    socket.send(JSON.stringify({ status: "success", msg: "Chat Notif Sent" }));
                    break;

                default:
                    console.log("Unknown notification type");
                        }

        } catch(e){ socket.error("Invalid JSON format received");
                }

});

console.log('WebSocket Server running at ws://0.0.0.0:8002');
```

Hasil :


localhost:8002
Save
Share
localhost:8002
Disconnect
Docs
Message
Params
Headers
Settings
Cookies
{
"type": "NEW_ASSIGNMENT",
"title": "AI Data Processing",
"dueDate": "2026-05-01"
}
Text
Send
Response
Connected
Search
All Messages
Clear Messages
{"status":"success","msg":"Assignment Notif Sent"}
11:35:57.621
{ "type": "NEW_ASSIGNMENT", "title": "AI Data Processing", "dueDate": "2026-05-01" }
11:35:57.614
{"status":"success","msg":"Chat Notif Sent"}
11:34:41.235
{ "type": "CHAT_MESSAGE", "sender": "Adif", "text": "Hey, have you finished the assignment?" }
11:34:41.216